Data Mining Assignment 3

Clustering and Anomaly Detection on a Mixed Article Dataset

Samuel van Nimwegen

1 May 2026

GitHub repository: https://github.com/samuelvnimwegen/data-mining-anomaly-detection.git

1 Task 1: Data Exploration and Preprocessing

1.1 Dataset overview

The dataset contains 2,164 documents stored in `articles.csv`. Each document is a short online discussion article. Based on an initial inspection of a random sample, I expected to find topic groups such as medicine, computing, space and science, sports, and car discussions, along with a small number of spam or corrupted files. Text length varies a lot, with an average of 886 characters and around 160 words per document. The sample also showed me that 82 documents contained HTML markup and 136 showed unusually high amounts of symbols, suggesting that some documents have an unusual structure.

1.2 Preprocessing pipeline

Because clustering and anomaly detection need different types of information, I made two separate versions of each document.

The clustering view cleans the text heavily: HTML, URLs, digits, and punctuation are removed, text is lowercased, stop-words are removed (NLTK), and words are reduced to their base forms using spaCy. Rare tokens (document frequency < 1) and very common tokens (document frequency $> 92\%$) are also removed. After this step, the vocabulary shrank from 345,517 to 160,129 tokens, a 54% reduction. The cleaned texts were turned into TF-IDF vectors (unigrams and bigrams, 20,000 features). I also built a bag-of-words matrix (unigrams, 20,000 features) from the clustering view and saved it to `data/results/bag_of_words_matrix.csv` as the Task 1 deliverable.

The anomaly view keeps punctuation, digits, and casing, so that repetition and formatting patterns are still visible. From this view, I built two feature sets: (i) a character n-gram TF-IDF matrix (3 to 5-grams, 30,000 features) reduced to 256 dimensions using LSA, and (ii) 17 custom structural features described in Section 3.

2 Task 2: Clustering

2.1 Methods and experimental design

I compared two ways of representing the documents across $k \in \{2, \ldots, 10\}$ using three algorithms. TF-IDF gives a sparse word matrix (unigrams and bigrams, 20,000 features). Sentence embeddings

are dense 384-dimensional vectors from `all-MiniLM-L6-v2`, which capture meaning rather than just matching words. The three algorithms I tried were K-Means (Euclidean distance), Agglomerative clustering (Ward linkage, which reduces the spread within each cluster at every step and tends to give more even cluster sizes than single or complete linkage), and Spectral clustering (RBF affinity kernel).

I used the silhouette score as the main number to compare results. As the assignment notes, a number alone is not enough to judge clustering quality, so I also looked at representative documents and the top terms per cluster to check that the groups make sense. I also added a size limit: the largest cluster must not have more than 1,300 documents (60% of the dataset), to avoid bad results where almost all documents end up in one group.

2.2 Results and cluster interpretation

Table 1 shows the results. Spectral clustering always broke the size limit (the RBF kernel puts most documents into one group) so I dropped it. TF-IDF struggles with so many features: cluster boundaries become unclear in that large sparse space, and the best TF-IDF result (Agglomerative $k = 8$, silhouette 0.053) has a very uneven size distribution with one cluster of 1,124 documents and others with only 3 to 7. Sentence embeddings fix this by working in a smaller dense space that captures meaning. Every embedding setup I tried scored higher than any TF-IDF option, with much more balanced cluster sizes (50 to 564 documents).

Effect of cluster count: going from $k = 5$ to $k = 8$ splits the large general-discussion group into more specific topics. Car discussions and general discussion get their own clusters, and a separate Food & diet group appears. Below $k = 5$ the spam documents start to mix with general discussion and the clean separation visible in Cluster 4 is lost. Above $k = 8$ the silhouette score drops and very small clusters of fewer than 30 documents appear, which are too small to be useful. I found that $k = 8$ gives the best balance between detail and stability.

I chose Embedding K-Means $k = 8$ (silhouette 0.062) for the submission over $k = 5$ (silhouette 0.065) because it finds more specific topic groups.

Table 1: Selected clustering results. = passes size constraint ($\leq$ 1300 documents). Best valid solution highlighted.

Method	k	Silhouette	Max cluster	Passes constraint
Embedding + K-Means	**5**	**0.065**	**510**	
Embedding + K-Means	8	0.062	397	
Embedding + Agglomerative	6	0.058	564	
TF-IDF + Agglomerative	8	0.053	1124	
TF-IDF + Agglomerative	7	0.069	1629	×
TF-IDF + K-Means	8	0.013	866	
TF-IDF + Spectral	9	0.092	2015	×

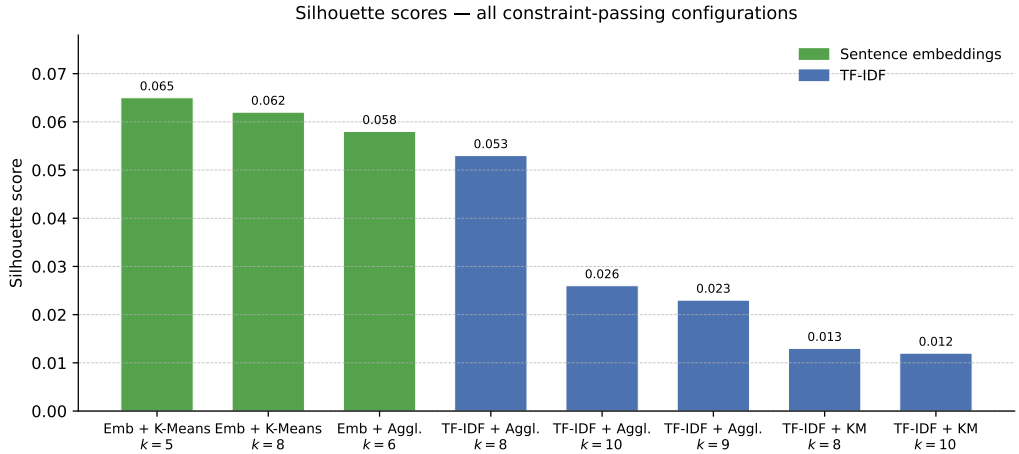

Figure 1: Silhouette scores for all setups that pass the size limit. Spectral clustering is left out.

Table 2 shows the eight clusters from the chosen solution. All clusters match recognisable topics. One thing worth noting: a quick TF-IDF check of Cluster 0 showed terms like "gordon bank" and "chastity", which look like spam. These come from a repeating email signature used by one frequent poster in the medical discussions. The sentence embeddings are not affected by this and correctly group those documents as medical discussions, which shows the advantage of dense vectors over a simple word count. The known anomalies are very concentrated: 41 of the 50 flagged documents are in Cluster 4 (Promotional spam), showing that my clustering and anomaly detection are finding the same pattern.

Table 2: Embedding K-Means $k = 8$ clusters: sizes and top terms.

Cluster	Inferred topic	Docs	Top content terms
0	Medicine & health	307	doctor, patient, disease, pain, medical, treatment
1	Automotive	374	car, engine, dealer, price, good
2	Graphics & computing	340	file, graphic, program, image, format, window
3	Space & science	236	space, moon, launch, nasa, orbit, lunar
4	Promotional spam	50	ref, promo, sale, id, offer, code
5	General discussion	362	just, like, think, point, time, know
6	Sports (hockey)	397	game, team, hockey, player, play, playoff
7	Food & diet	98	food, msg, taste, cause, chinese, diet

To check the results beyond just the number, I looked at the document closest to each cluster centre as a representative example. Cluster 6's example discusses NHL playoff scheduling. Cluster 3's example is about the cost of lunar missions. Cluster 7's example debates food poisoning. Cluster 4's example is a short promotional listing with repeated offer codes, which fits the top terms exactly. These examples confirm that the clusters reflect real topic groups and are not just noise.

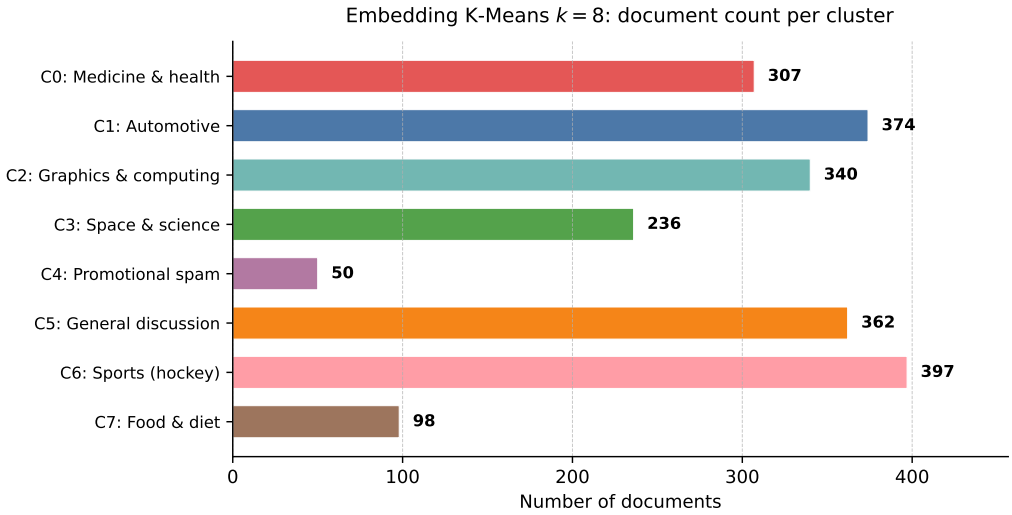

Figure 2: Document counts per cluster for the $k = 8$ solution (50 to 397 documents per cluster).

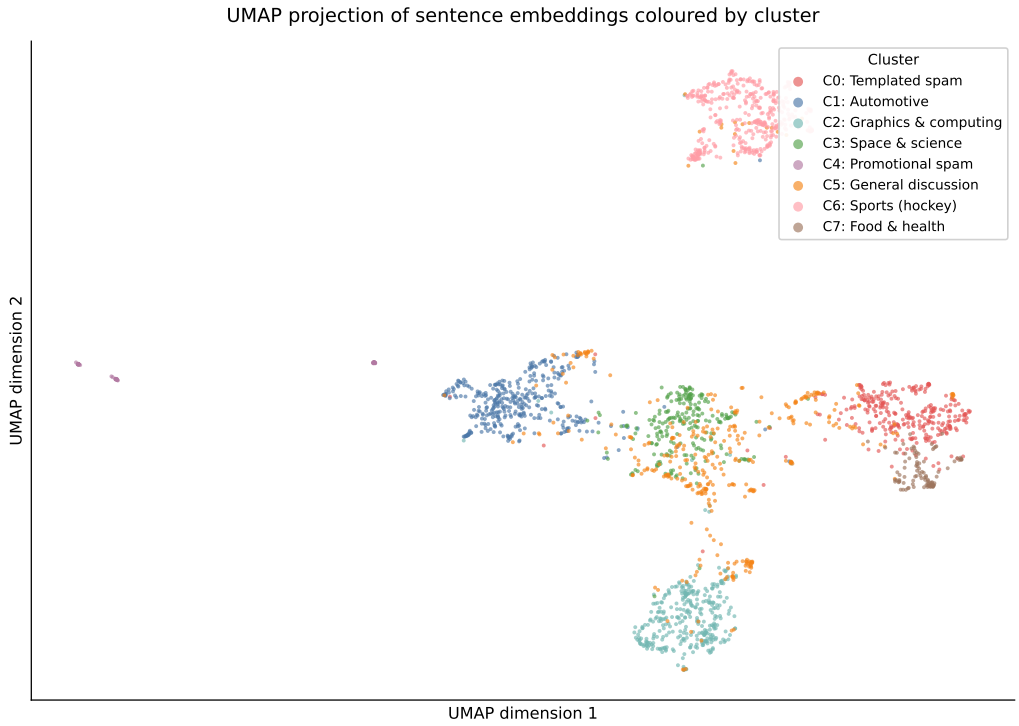

Figure 3: UMAP projection of the 384-dimensional sentence embeddings coloured by cluster. The spam cluster (C4) is clearly separated on the left. Graphics (C2) is compact and distinct. General discussion (C5) sits between Sports and Cars, as expected for a mixed-topic group.

3 Task 3: Anomaly Detection

3.1 Feature engineering

Documents that are unusual because of formatting or repetition are hard to find with features based on word meaning. I created 17 custom structural features, covering: length (character count, token count, average token length), vocabulary (type-token ratio, repeated-token ratio, max repeated-token run, lexical entropy, bigram type-token ratio), character-class ratios (upper-

case, digit, punctuation), special patterns (URL count, email count, HTML tag count, mixed alphanumeric ratio, placeholder ratio), and compression ratio (compressed size divided by original size), which directly measures how repetitive a document is. This gives a feature matrix of shape $(2164, 17)$.

3.2 Methods and results

I ran three detectors on the structural feature matrix. Isolation Forest (IF) uses a contamination of 2.3% and scores each document by how easily it can be separated from the rest using random splits. Local Outlier Factor (LOF) uses $k = 20$ neighbours and scores each document by how dense its neighbourhood is compared to its neighbours. kNN uses the average distance to the 10 nearest neighbours.

Table 3 shows how much the top-50 lists overlap. IF and LOF agree on 22 of 50 (44%). kNN is very different, sharing only 2 documents with each of the others. The reason is the clustered anomaly problem: the 50 repeated junk documents are close to each other, so their average distance to neighbours is small. kNN therefore ranks them as normal and marks ordinary documents as suspicious instead. LOF has the same problem but is a bit less affected. Isolation Forest is not fooled because it measures how easy it is to isolate a document, not distance.

I also tested word-based features (TF-IDF and LSA) with Isolation Forest. The Spearman correlation between their rankings was -0.131, with only 4 shared candidates in the top 50, showing that the unusual documents stand out in structure, not in the words they use.

Table 3: Pairwise top-50 overlap between anomaly detection methods (structural features).

Method pair	Top-50 overlap
IF vs. LOF	22 / 50
IF vs. kNN	2 / 50
LOF vs. kNN	2 / 50

3.3 Final anomaly list

I chose Isolation Forest on the 17 structural features as the main detector and saved the 50 top-ranked documents to `anomalies.csv`. The difference between anomalies and normal documents is clear: the median compression ratio for anomalies is 0.43 versus 0.63 for normal documents, and the median repeated-token ratio is 0.83 versus 0.42. Figure 4 shows this across four features. The top candidates include documents made of a single repeated phrase (e.g. "fast deal fast deal…"), dense promotional text with markup fragments, and heavily quoted reply chains with a lot of punctuation.

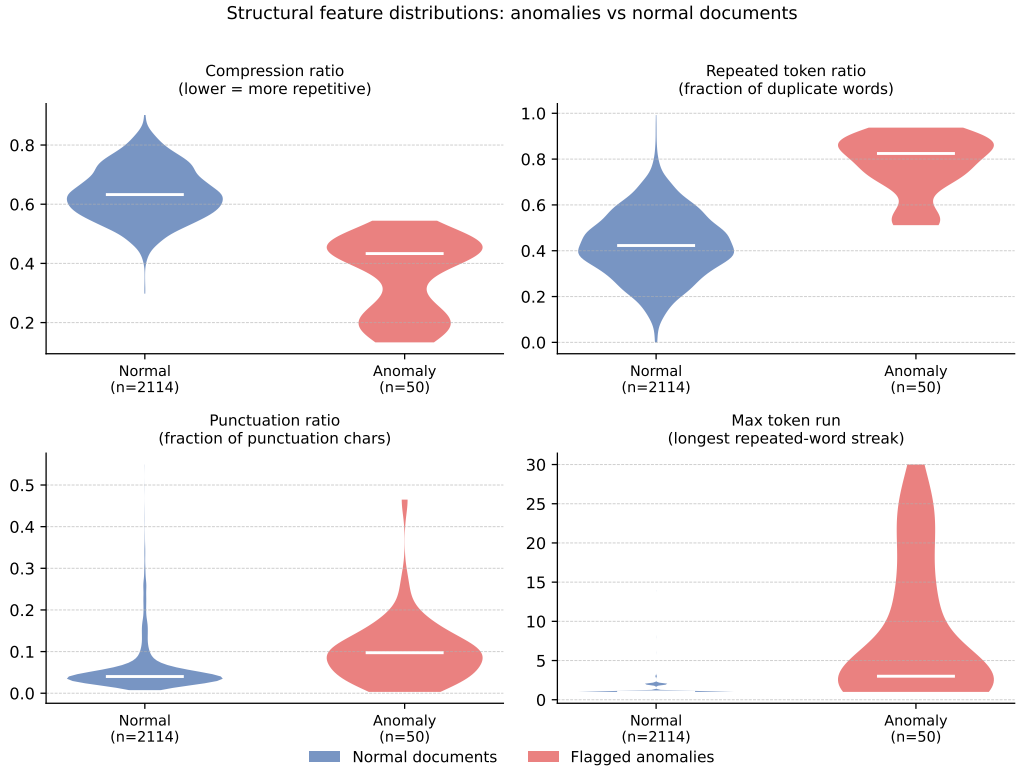

Figure 4: Violin plots comparing four structural features for the 50 anomalies (red) versus 2,114 normal documents (blue). Anomalies have lower compression ratios, higher repeated-token ratios, higher punctuation ratios, and much larger max token runs (capped at 30 for display).

4 Conclusion

For clustering, I found that sentence embeddings (`all-MiniLM-L6-v2`) work much better than TF-IDF across all methods I tried. My final submission uses Embedding K-Means $k = 8$ (silhouette 0.062), which gives eight clear topic groups. The UMAP plot confirms that these groups are well separated in the embedding space. For anomaly detection, I found that Isolation Forest on 17 structural features does better than both distance-based methods (kNN, LOF) and word-based methods (TF-IDF, LSA). The violin plots clearly show that the 50 flagged documents stand apart from the 2,114 normal documents on every structural feature I used.